

পিসি-বিহীন অ্যান্ড্রয়েড এআই অ্যাপ্লিকেশন ইঞ্জিন ও স্বয়ংক্রিয় এপিকে জেনারেশন ক্লপিং

সফটওয়্যার ইঞ্জিনিয়ারিংয়ের আধুনিক রূপান্তর এখন আর কেবল স্ট্যাটিক কোড জেনারেশনের মধ্যে সীমাবদ্ধ নেই, বরং এটি একটি স্বয়ংক্রিয় মাল্টি-এজেন্ট সিস্টেমে পরিণত হয়েছে যা মূলত আইসোলেটেড রানটাইম এনভায়রনমেন্টে কাজ করতে সক্ষম। ল্যাপটপ বা ডেস্কটপ কম্পিউটারের মতো প্রথাগত মাধ্যম এড়িয়ে সম্পূর্ণ অ্যান্ড্রয়েড মোবাইল ডিভাইস দিয়ে নেটিভ অ্যান্ড্রয়েড অ্যাপ্লিকেশন তৈরির এই পরিকল্পনাটি একটি যুগান্তকারী প্রযুক্তিগত ক্লপিং। এই বিশ্লেষণে গুগলের জুলস (Jules) প্ল্যাটফর্ম, মডেল কনটেক্সট প্রোটোকল (MCP) এবং অন্যান্য সহযোগী ক্লাউড ও অন-ডিভাইস টুলগুলির সমন্বয়ে একটি সম্পূর্ণ স্বয়ংক্রিয় 'প্রাইভেট ফ্যাক্টরি' গড়ে তোলার কৌশলগত কর্মপরিকল্পনা উপস্থাপন করা হয়েছে। গুগল ক্লাউড প্ল্যাটফর্ম (GCP) বা ফায়ারবেস স্টুডিওর প্রথাগত বিলিং জটিলতা এবং ক্রেডিট কার্ড নিবন্ধনের ঝামেলা সম্পূর্ণরূপে এড়িয়ে কনজিউমার এপিআই কী (Consumer API Key) ব্যবহারের মাধ্যমে এই সম্পূর্ণ ইকোসিস্টেমটি কোনো প্রকার খরচ ছাড়াই পরিচালনা করা সম্ভব। নিচে এই সিস্টেমটির একটি তুলনামূলক প্রযুক্তিগত বিশ্লেষণ উপস্থাপন করা হলো।

স্থানীয় বনাম ক্লাউড-ভিত্তিক ইঞ্জিন কৌশলগত মূল্যায়ন

একটি মোবাইল-কেন্দ্রিক এআই অ্যাপ্লিকেশন ইঞ্জিন তৈরি করতে হলে দুটি প্রধান কৌশলগত মডেল বিশ্লেষণ করা আবশ্যিক: স্থানীয় টার্মাক্স (Termux) ভিত্তিক ইঞ্জিন এবং ক্লাউড-ভিত্তিক স্বয়ংক্রিয় এআই ইন্টিগ্রেশন মডেল। প্রথাগত ফায়ারবেস স্টুডিও বা গুগল ক্লাউড এআই স্টুডিও ব্যবহারের জন্য যেখানে একটি সক্রিয় পেমেন্ট গেটওয়ে এবং কার্ডের প্রয়োজন হয়, সেখানে এই বিকল্প কর্মপদ্ধতি সম্পূর্ণ বিনামূল্যে এআই-নিয়ন্ত্রিত ডেভেলপমেন্ট বাস্তবায়ন করে।

মূল্যায়নের ক্ষেত্র	ফায়ারবেস স্টুডিও / ক্লাউড এআই বিল্ডার	স্থানীয় টার্মাক্স ইঞ্জিন (অফলাইন/অন-ডিভাইস)	কাস্টম জুলস + এমসিপি হাইব্রিড ক্লাউড ফ্যাক্টরি
বিলিং বাধ্যবাধকতা	সক্রিয় জিসিপি বিলিং অ্যাকাউন্ট এবং কার্ডের সত্যতা নিশ্চিতকরণ আবশ্যিক।	সম্পূর্ণ নিষ্ক্রিয়; কোনো ক্লাউড বিলিং বা সংযোগের প্রয়োজন নেই।	জিসিপি বিলিং এড়িয়ে কনজিউমার এপিআই কী-এর সাহায্যে সম্পূর্ণ বিনামূল্যে পরিচালিত।
হার্ডওয়্যার সক্ষমতা ও থ্রটলিং	শূন্য স্থানীয় লোড; সমস্ত ভারী প্রসেস গুগলের হাই-এন্ড ক্লাউড সার্ভারে সম্পন্ন হয়।	প্রসেসর এবং মেমরির ওপর অতিরিক্ত চাপ; পোকো এক্স৩ (POCO X3) ফোনে থ্রটলিং ও ওওএম (OOM) কিলারের ঝুঁকি।	পোকো এক্স৩ শুধুমাত্র রিমোট কন্ট্রোল হিসেবে কাজ করে; কম্পাইলেশন সম্পন্ন হয় ট্রানজিয়েন্ট ভার্চুয়াল মেশিনে।
কনটেক্সট উইন্ডো এবং যুক্তি সক্ষমতা	মিডিয়াম টু হাই ($128 \times K$) থেকে $1 \times M$ টোকেন)।	অত্যন্ত সীমিত; স্থানীয় ওলামা (Ollama) মডেলে সর্বোচ্চ $8 \times K$ টোকেন স্কেলিং সম্ভব।	সর্বোচ্চ 2×10^6 (২ মিলিয়ন) টোকেন সমৃদ্ধ জেমিনাই ২.৫ প্রো এবং জেমিনাই ৩ ইঞ্জিন।
টুলিং ও এক্সটার্নাল এপিআই অ্যাক্সেস	প্রাক-নির্ধারিত গুগল লাইব্রেরি এবং টেমপ্লেটের মধ্যে সীমাবদ্ধ।	টার্মাক্স-বান্ধব বাইনারি এবং মৌলিক স্ক্রিপ্টিং টুলের মধ্যে সীমাবদ্ধ।	এমসিপি সার্ভারগুলোর মাধ্যমে সরাসরি ডেটাবেস ও ডিজাইন সিস্টেমে নিরবচ্ছিন্ন সংযোগ।
বিল্ড স্থায়িত্ব ও ব্যর্থতার হার	উচ্চমানের স্থায়িত্ব কিন্তু উচ্চ আইনি ও পেমেন্ট জটিলতা।	উচ্চ ব্যর্থতার হার; টার্মাক্সে ডাইনামিক লিংকার ও লাইব্রেরি পাথের অসঙ্গতির কারণে বিল্ড ক্র্যাশ হতে পারে।	অত্যন্ত উচ্চমানের স্থায়িত্ব; স্ট্যান্ডার্ড গিটহাব অ্যাকশন রানারে পরিচালিত স্বয়ংক্রিয় ডকার বিল্ড।

এই প্রযুক্তিগত তুলনা থেকে স্পষ্ট প্রমাণিত হয় যে, পোকো এক্স৩ (POCO X3) ফোনের মতো মোবাইল ডিভাইসে সম্পূর্ণ স্থানীয়ভাবে কোড কম্পাইল করা হলে তা সিস্টেমের মেমরি এবং প্রসেসিং ক্ষমতার চরম সীমাতে পৌঁছায়। কম্পাইল প্রক্রিয়ার জন্য প্রয়োজনীয় গাণিতিক শক্তির সমীকরণটি নিম্নরূপ:

$$\text{Required Execution Memory} \geq 1.5 \text{ GB} + \text{Android Kernel Overhead} + \text{Gradle Daemon Allocations}$$

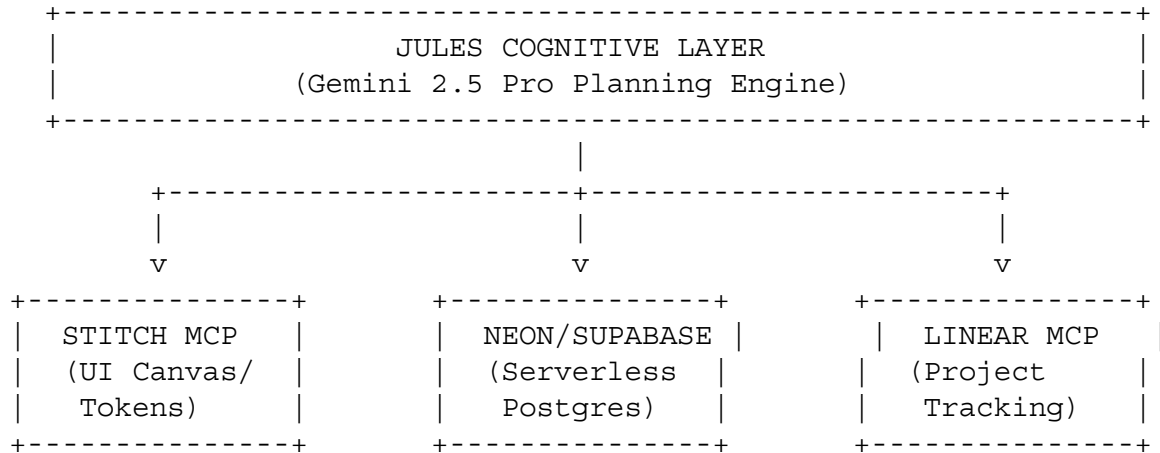
অতএব, স্থানীয় টারমাক্সকে একটি সমন্বয়কারী এবং রিমোট কন্ট্রোল রুম হিসেবে ব্যবহার করে ক্লাউড-ফার্স্ট স্বায়ত্তশাসিত মডেল (গিটহাব রানার ও জুলস ভিএম) বাস্তবায়ন করাই প্রকৌশলী নির্ধারণের জন্য সবচেয়ে যুক্তিযুক্ত এবং কার্যকর সমাধান।

জুলস এবং মডেল কনটেক্সট প্রোটোকল ইন্টিগ্রেশন বিশ্লেষণ

গুগলের জুলস প্ল্যাটফর্ম মূলত একটি স্বায়ত্তশাসিত এবং অ্যাসিঙ্ক্রোনাস কোডিং এজেন্ট যা ডেভেলপারকে কোড লেখার বাস্তব প্রক্রিয়া থেকে সম্পূর্ণরূপে মুক্ত করে। এটি সরাসরি গুগলের ট্রানজিয়েন্ট ক্লাউড ভার্সুয়াল মেশিনে রান করে এবং মডেল কনটেক্সট প্রোটোকল (MCP) এর মাধ্যমে বিভিন্ন বাহ্যিক সার্ভার এবং টুলস নিয়ন্ত্রণ করতে পারে।

জুলস প্ল্যাটফর্মের প্রিমিয়াম অ্যাকাউন্টের আওতায় থাকা কৌশলগত সুবিধাগুলো ব্যবহার করে একটি সম্পূর্ণ 'পকেট এআই অ্যাপ ফ্যাক্টরি' গড়ে তোলা সম্ভব। ব্যবহারকারীকে প্রথমে jules.google.com/settings এ গিয়ে নিম্নলিখিত অফিসিয়াল এমসিপি (MCP) সার্ভারগুলো সংযুক্ত করতে হবে:

- v0** (বাই ভেরসেল) অথবা **Stitch** (বাই গুগল): এটি অ্যাপ্লিকেশনের রেসপনসিভ ফ্রন্টএন্ড লেআউট এবং ইউজার ইন্টারফেস (UI) ডিজাইন তৈরি করতে ব্যবহৃত হয়। এটি সরাসরি ভিজুয়াল ডিজাইন মেটাডেটাকে স্ট্যান্ডার্ড রিয়াক্ট বা HTML5 কোডে রূপান্তর করে।
- Supabase** অথবা **Neon**: এটি একটি সার্ভারলেস পোস্টগ্রেস (Postgres) ডেটাবেস যা অ্যাপ্লিকেশনের ব্যাকএন্ড ডেটা সংরক্ষণ, ব্যবহারকারী প্রমাণীকরণ এবং ডেটা রিলেশনশিপ ম্যানেজমেন্টের জন্য কাজ করে।
- Linear** (ইন্ট্রিক): এটি বৃহৎ আকারের অ্যাপ্লিকেশনের ক্ষেত্রে পর্যায়ক্রমিক কার্যতালিকা এবং বাগ ট্র্যাকিং পরিচালনার জন্য সহায়ক ভূমিকা পালন করে।
- Context7**: এটি সেশন পরিচালনা এবং মেটাডেটা ট্র্যাকিংয়ের অতিরিক্ত কনটেক্সট সরবরাহ করতে সাহায্য করে।



কর্মক্ষমতা বিভাজন: স্বয়ংক্রিয় বনাম ম্যানুয়াল কনফিগারেশন

এই আর্কিটেকচারটি সফলভাবে পরিচালনা করার জন্য কোন কনফিগারেশনগুলো জুলস নিজে স্বয়ংক্রিয়ভাবে করতে পারে এবং কোনগুলো প্রকৌশলীকে ম্যানুয়ালি সম্পাদন করতে হবে, তা নির্ধারণ করা আবশ্যিক।

কর্মক্ষেত্র ও উপাদান	ম্যানুয়াল কনফিগারেশন (ব্যবহারকারীর কাজ)	স্বয়ংক্রিয় সম্পাদন (জুলস এআই-এর কাজ)
অ্যাকাউন্ট ও ইন্টিগ্রেশন অথরাইজেশন	বাহ্যিক প্ল্যাটফর্মের (Stitch, Neon, GitHub) এপিআই কী এবং ওঅথ২ (OAuth2) অনুমোদন ম্যানুয়ালি সম্পন্ন করা।	অনুমোদিত সংযোগের মাধ্যমে বাহ্যিক রানটাইম লাইব্রেরি অ্যাক্সেস করা এবং কাজের সমন্বয় সাধন করা।
আধার এবং সংগ্রহস্থল (Repository) তৈরি	গিটহাবে একটি নতুন রিপোজিটরি তৈরি করা এবং টারমাক্সের সাথে সংযোগ স্থাপন করা।	নির্দেশ পাওয়ার পর রিপোজিটরি ক্লোন করা এবং সেখানে নতুন কোড মডিউল যোগ করা।
প্রয়োজনীয় সিক্রেট সংরক্ষণ	গিটহাব সিক্রেটস (GitHub Secrets) বিভাগে সাইনিং কী এবং ফায়ারবেস ডিস্ট্রিবিউশন টোকেনগুলো সংরক্ষণ করা।	সিক্রেটের সাথে কোনো প্রকার অসঙ্গতি ছাড়াই নিরাপদ কোড তৈরি এবং ডিপ্লয়মেন্ট স্ক্রিপ্ট রান করা।
ফ্রন্টএন্ড এবং ব্যাকএন্ড ডিজাইন	একটি সাধারণ টেক্সট প্রম্পট, ক্লিপিং বা অ্যাপের আইডিয়া ফাইল আকারে আপলোড করা।	v0 বা Stitch এর মাধ্যমে UI কোড ও manifest.json তৈরি এবং Neon ডেটাবেসে রিলেশনাল টেবিল ডিজাইন করা।
সিআই/সিডি সিস্টেম এবং রানার তৈরি	গিটহাব অ্যাকশনের জন্য প্রাথমিক ওয়ার্কফ্লো YAML ফাইল এবং বাবলরাপ (Bubblewrap) রুলস সেটআপ করা।	বাগ সনাক্তকরণ, কোডের ত্রুটি সম্পাদন, পুল রিকোয়েস্ট (PR) পাঠানো এবং স্বয়ংক্রিয়ভাবে সিআই পাইপলাইন সক্রিয় করা।

ক্লিক-বাই-ক্লিক রেডি-টু-ইউজ গাইডলাইন

নিম্নে সম্পূর্ণ অ্যাপ্লিকেশন ইঞ্জিন এবং জেনারেটর সিস্টেমটি সেটআপ করার ধাপে ধাপে নির্দেশিকা দেওয়া হলো।

ধাপ ১: গিটহাব রিপোজিটরি এবং সিক্রেট কনফিগারেশন

- আপনার মোবাইল ব্রাউজার থেকে github.com এ প্রবেশ করে একটি নতুন ব্যক্তিগত রিপোজিটরি তৈরি করুন (যেমন: android-app-factory)।
- রিপোজিটরির **Settings > Secrets and variables > Actions** মেনুতে যান।
- New repository secret** বাটনে ক্লিক করে নিম্নলিখিত সিক্রেটগুলো যুক্ত করুন :
 - SIGNING_KEY**: আপনার অ্যান্ড্রয়েড সাইনিং কী-এর Base64 কোড (এটি টারমাক্স থেকে তৈরি করে নিতে হবে)।
 - KEYSTORE_PASSWORD**: সাইনিং কী-এর পাসওয়ার্ড।
 - [span_38](start_span)[span_38](end_span)KEY_ALIAS**: কী-এর নাম (সাধারণত key0 হয়)।
 - [span_39](start_span)[span_39](end_span)KEY_PASSWORD**: কী-এর পাসওয়ার্ড।
 - [span_40](start_span)[span_40](end_span)FIREBASE_APP_ID**: ফায়ারবেস কনসোলার প্রোজেক্ট সেটিংস থেকে প্রাপ্ত অ্যান্ড্রয়েড অ্যাপ আইডি।
 - [span_41](start_span)[span_41](end_span)FIREBASE_SERVICE_ACCOUNT_JSON**: গুগল ক্লাউড কনসোল থেকে প্রাপ্ত ফায়ারবেস ডিস্ট্রিবিউশন সার্ভিস অ্যাকাউন্ট জেডিকে (JSON) ফাইলের সম্পূর্ণ টেক্সট।

ধাপ ২: জুলস প্ল্যাটফর্মে এমসিপি সংযোগ

- jules.google.com ওয়েবসাইটে গিয়ে আপনার প্রিমিয়াম গুগল অ্যাকাউন্ট দিয়ে লগইন করুন।
- ডানদিকের কোণায় থাকা প্রোফাইল ছবিতে ক্লিক করে **Settings > MCP** বিভাগে যান।

3. **Neon MCP** এবং **Stitch MCP** এর পাশে থাকা Connect বাটনে ক্লিক করে সংশ্লিষ্ট প্ল্যাটফর্মগুলোর সেটিংস থেকে আনা এপিআই কী পেস্ট করে যুক্ত করুন।
4. একই পেজে গিটহাবের সাথে জুলসের সংযোগ নিশ্চিত করুন এবং আপনার তৈরি করা রিপোজিটরির অ্যাক্সেস অনুমোদন করুন।

ধাপ ৩: পোকো এক্স৩-তে টারমাক্স লোকাল সেটআপ

1. গুগল প্লে স্টোর এডিয়ে অফিসিয়াল F-Droid বা গিটহাব রিলিজ থেকে টারমাক্স অ্যাপটি ফোনে ডাউনলোড করে ইনস্টল করুন।
2. টারমাক্স ওপেন করে ডিভাইসের স্টোরেজ অ্যাক্সেস চালু করতে নিচের কমান্ডটি রান করুন :
`termux-setup-storage`
3. এই গাইডের পরবর্তী বিভাগে দেওয়া স্বয়ংক্রিয় টারমাক্স সেটআপ স্ক্রিপ্টটি ডাউনলোড করে রান করুন।

ধাপ ৪: জুলস-টু-গিটহাব সিআই/সিডি অটোমেশন রান

1. টারমাক্সে আপনার তৈরি অ্যাপ্লিকেশনটির নকশা বা আইডিয়া সমৃদ্ধ একটি টেক্সট ফাইল তৈরি করুন (যেমন: `blueprint.txt`)।
2. টারমাক্স এপিআই স্ক্রিপ্টটি চালু করুন। এটি জুলসের এপিআই-কে কল করে রানটাইম সেশন তৈরি করবে।
3. জুলস স্বয়ংক্রিয়ভাবে কোড এবং ব্যাকএন্ড টেবিল তৈরি করে রিপোজিটরির main ব্রাঞ্চে পুশ করবে।
4. পুশ হওয়ার সাথে সাথে গিটহাব অ্যাকশন পাইপলাইন চালু হবে, বাবলর্যাপ দিয়ে কোডটিকে এপিকেতে রূপান্তর করবে এবং ফায়ারবেস অ্যাপ ডিস্ট্রিবিউশনের মাধ্যমে আপনার ফোনে পাঠাবে। আপনি ফায়ারবেসের ইমেল বা নোটিফিকেশনের মাধ্যমে সরাসরি ইনস্টল লিঙ্কটি পেয়ে যাবেন।

রেডি-টু-ইউজ ম্যাটেরিয়ালস, সোর্স কোড ও প্রম্পট

এই সেকশনে প্রয়োজনীয় প্রতিটি স্ক্রিপ্ট এবং প্রম্পটের সম্পূর্ণ প্রস্তুত কোড সরবরাহ করা হয়েছে, যা কোনো পরিবর্তন ছাড়াই সরাসরি ব্যবহারযোগ্য।

১. টারমাক্স স্বয়ংক্রিয় এনভায়রনমেন্ট সেটআপ স্ক্রিপ্ট

টারমাক্সে জাভা ২১, এনডিকে, এবং অ্যান্ড্রয়েড কমান্ড-লাইন টুলস সেটআপ করার জন্য রানযোগ্য সম্পূর্ণ স্ক্রিপ্ট :

```
#!/data/data/com.termux/files/usr/bin/bash
```

```
echo "[*] টারমাক্স প্যাকেজ সোর্স আপডেট করা হচ্ছে..."
```

```
pkg update -y && pkg upgrade -y
```

```
echo "[*] প্রয়োজনীয় ডেভলপমেন্ট প্যাকেজসমূহ ইনস্টল করা হচ্ছে..."
```

```
pkg install wget curl git openjdk-21 nodejs lsof unzip coreutils -y
```

```
echo "[*] অ্যান্ড্রয়েড এসডিকে (Android SDK) ডাউনলোডার চালু করা হচ্ছে..."
```

```
wget -O ~/install-android-sdk.sh
```

```
https://raw.githubusercontent.com/Sohil876/termux-sdk-installer/main/installer.sh
```

```
chmod +x ~/install-android-sdk.sh
```

```
bash ~/install-android-sdk.sh
```

```
# এনভায়রনমেন্ট ভেরিয়েবল সেটআপ করা
echo 'export ANDROID_HOME=$HOME/android-sdk' >> ~/.bashrc
echo 'export
PATH=$PATH:$ANDROID_HOME/cmdline-tools/latest/bin:$ANDROID_HOME/platfo
rm-tools' >> ~/.bashrc
source ~/.bashrc

echo "[*] এসডিকে লাইসেন্স এবং অ্যান্ড্রয়েড ৩৫ প্ল্যাটফর্ম ইনস্টল করা হচ্ছে..."
yes | sdkmanager --licenses
yes | sdkmanager "platforms;android-35" "build-tools;35.0.0"

echo "[*] গ্লোবাল গ্রেডল প্রোপার্টিজ ফাইলে aapt2 ওভাররাইড যুক্ত করা হচ্ছে..."
mkdir -p ~/.gradle
echo
"android.aapt2FromMavenOverride=/data/data/com.termux/files/usr/bin/aa
pt2" > ~/.gradle/gradle.properties

echo "[✓] স্থানীয় টারমাক্স এনভায়রনমেন্ট সফলভাবে প্রস্তুত করা হয়েছে!"
```

২. শিয়ুকু-রিশ স্বয়ংক্রিয় স্কোপড স্টোরেজ ডেটা সিলেক্ট স্ক্রিপ্ট

অ্যান্ড্রয়েডের ডেটা ফোল্ডারের ফাইল সীমাবদ্ধতা এড়িয়ে শিয়ুকু (rish) এর মাধ্যমে ডেভেলপমেন্ট ডেটা নিরাপদে স্থানান্তর করার স্বয়ংক্রিয় স্ক্রিপ্ট:

```
#!/data/data/com.termux/files/usr/bin/bash

# টারমাক্স স্যান্ডবক্সে টেম্পোরারি ওয়ার্কিং জোন তৈরি
STAGING_DIR="/data/data/com.termux/files/home/workspace_staging"
mkdir -p "$STAGING_DIR"

echo "[*] শিয়ুকু (rish) ব্যবহার করে সুরক্ষিত ডেটা রিড করা হচ্ছে..."
# ডাউনলোডের ডিরেক্টরিতে ডেটা কপি করা হচ্ছে
/data/data/com.termux/files/usr/bin/rish -c "cp -r
/storage/emulated/0/Android/data/com.fact.app/files/src/*
/sdcard/Download/temp_app_src/"

# শেয়ার্ড জোন থেকে ফাইলগুলো টারমাক্স স্যান্ডবক্সে স্থানান্তর
mv /sdcard/Download/temp_app_src/* "$STAGING_DIR/"

echo "[*] ফাইলগুলোর এমডি৫ হ্যাশ পরীক্ষা করে ডুপ্লিকেট রিমুভ করা হচ্ছে..."
find "$STAGING_DIR" -type f -exec md5sum {} + | sort | awk 'BEGIN
{last=""} {if($1==last) print $2; last=$1}' | xargs rm -f

echo "[*] কোডবেস জিপ করা হচ্ছে এবং টেম্প জোন পরিষ্কার করা হচ্ছে..."
zip -r9 "$HOME/processed_app.zip" "$STAGING_DIR"
rm -rf "$STAGING_DIR"
rm -rf /sdcard/Download/temp_app_src
```

```
echo "[✓] ডেটা প্রসেসিং সমাপ্ত! আউটপুট ফাইল: $HOME/processed_app.zip"
```

৩. ক্যাপাসিটর বাবলর্যাপ প্রজেক্ট রানিং গাইড

রিয়াক্ট বা নেক্সট.জেএস প্রজেক্টের কোড সরাসরি এপিকে-তে রূপান্তর করার পদ্ধতি :

```
# ১. প্রথমত কোডটির স্ট্যাটিক এক্সপোর্ট তৈরি করুন (Next.js এর ক্ষেত্রে next.config.js  
ফাইলে output: 'export' কনফিগার করুন)  
npm run build
```

```
# ২. ক্যাপাসিটর অ্যান্ড্রয়েড প্ল্যাটফর্ম যুক্ত করা  
npx cap init "My AI App" "com.nirjhar.aiapp" --web-dir=out  
npx cap add android
```

```
# ৩. ওয়েব কোড সিঙ্ক করা  
npx cap sync
```

```
# ৪. গ্রেডল রিমোট রানার দিয়ে অফলাইনে বিল্ড করা  
cd android  
./gradlew assembleDebug --no-daemon
```

বাবলর্যাপ (*PWA wrapping*) পদ্ধতির জন্য গ্লোবাল রানার স্ক্রিপ্ট :

```
npm install -g @bubblewrap/cli  
bubblewrap init --manifest=https://your-pwa-url.com/manifest.json  
bubblewrap build
```

৪. গিটহাব অ্যাকশন পাইপলাইন প্রোডাকশন ওয়ার্কফ্লো (.github/workflows/main.yml)

গিটহাব রানারে বাবলর্যাপ দিয়ে অটোমেটেড বিল্ড এবং ফায়ারবেস অ্যাপ ডিস্ট্রিবিউশন ট্রিগার করার জন্য সম্পূর্ণ প্রস্তুত YAML ফাইল:

```
name: Build & Distribute signed Android APK
```

```
on:
```

```
  push:  
    branches: [ main ]  
  workflow_dispatch:
```

```
jobs:
```

```
  build-android:  
    runs-on: ubuntu-latest  
    steps:  
      - name: Checkout Code  
        uses: actions/checkout@v4  
  
      - name: Setup Node.js Runtime  
        uses: actions/setup-node@v4  
        with:
```

```

    node-version: '20'
    cache: 'npm'

- name: Install Bubblewrap CLI
  run: npm install -g @bubblewrap/cli

- name: Setup Java Development Kit (JDK 17)
  uses: actions/setup-java@v4
  with:
    distribution: 'temurin'
    java-version: '17'

- name: Restore Gradle Cache
  uses: actions/cache@v4
  with:
    path: |
      ~/.gradle/caches
      ~/.gradle/wrapper
    key: ${{ runner.os }}-gradle-${{ hashFiles('**/*.gradle*',
'**/gradle-wrapper.properties') }}

- name: Parse PWA Manifest and Setup Project
  run: |
    bubblewrap init
--manifest=https://raw.githubusercontent.com/${{ github.repository
}}/main/public/manifest.json --directory=./android-build --yes

- name: Build and Sign Production APK
  working-directory: ./android-build
  run: |
    echo "${{ secrets.SIGNING_KEY }}" | base64 -d > release.jks
    chmod +x ./gradlew
    ./gradlew assembleRelease \
      -Pandroid.injected.signing.store.file=release.jks \
      -Pandroid.injected.signing.store.password=${{
secrets.KEYSTORE_PASSWORD }} \
      -Pandroid.injected.signing.key.alias=${{ secrets.KEY_ALIAS
}} \
      -Pandroid.injected.signing.key.password=${{
secrets.KEY_PASSWORD }}

- name: Upload APK to Firebase App Distribution
  uses: wzieba/Firebase-Distribution-Github-Action@v1
  with:
    appId: ${{ secrets.FIREBASE_APP_ID }}
    serviceCredentialsFileContent: ${{
secrets.FIREBASE_SERVICE_ACCOUNT_JSON }}
    groups: testers

```

```
file:  
android-build/app/build/outputs/apk/release/app-release.apk
```

৫. জুলস মাস্টার অর্কেস্ট্রেটর এআই প্রম্পট

জুলসের কনসোলে রান করার জন্য সর্বোচ্চ আউটপুট সমৃদ্ধ রেডি-টু-ইউজ প্রম্পট :

আপনি এখন একজন "Master AI Software Architect & Senior Mobile Systems Engineer" হিসেবে কাজ করবেন।

প্রজেক্টের প্রেক্ষাপট: আমরা একটি পিসি-বিহীন স্বয়ংক্রিয় অ্যাপ্লিকেশন ইঞ্জিন তৈরি করছি যেখানে গিটহাব অ্যাকশন এবং বাবলরুম ব্যবহার করে ওয়েব অ্যাপকে নেটিভে রূপান্তর করা হবে।

আপনার নির্দিষ্ট দায়িত্বসমূহ: ১. রিপোজিটরির রুট ফাইলগুলো যাচাই করুন। যদি কোনো 'manifest.json' ফাইল না থাকে, তবে অবিলম্বে একটি সম্পূর্ণ স্ট্যান্ডার্ড PWA ম্যানিফেস্ট ফাইল তৈরি করুন যার ক্যাশ কনফিগারেশন এবং স্টার্ট ইউআরএল নির্খুঁত হবে। ২. ভিজুয়াল আর্কিটেকচারের জন্য 'Stitch MCP' ব্যবহার করুন এবং WCAG AAA অনুপাত মেনে চমৎকার ইন্টারফেসের কালার টোকেন ডিজাইন তৈরি করে 'Tailwind CSS' কনফিগার করুন। ৩. 'Neon' বা 'Supabase' ডেটাবেস এমসিপি ব্যবহার করে প্রয়োজনীয় পোস্টগ্রেস ডেটাবেস টেবিল এবং ইউজার অথেন্টিকেশন রিলেশনশিপ স্কিমা ডিজাইন করুন। ৪. ডেটাবেসের অ্যাক্সেস ক্রেডেনশিয়ালগুলো সুরক্ষিতভাবে অ্যাপ্লিকেশনের কনফিগারেশন মডিউলের মধ্যে ইনজেক্ট করুন। ৫. সম্পূর্ণ কোডবেস পরীক্ষা শেষে কোনো ত্রুটি না রেখে সরাসরি 'main' ব্রাঞ্চে পুশ করুন, যেন গিটহাব অ্যাকশনের কম্পাইলার এপিকে তৈরির কাজটি স্বয়ংক্রিয়ভাবে শুরু করতে পারে।

৬. এআই গাইডেন্স কনটেক্সট পিনিং ফাইল (AGENTS.md)

জুলস যেন নিজের সীমার বাইরে গিয়ে অপ্রয়োজনীয় কোড বা অগোছালো বিন্দু তৈরি না করে, তার জন্য রিপোজিটরির রুটে রাখার উপযোগী নির্দেশিকা :

Agent Directives for Android PWA Factory

Core Architectural Boundaries

- **Frontend Core:** Strictly Next.js / React formatted with continuous service-worker caching.
- **Database Scope:** Neon Serverless PostgreSQL with optimized database indexing.
- **Design Guidelines:** Stitch token integration utilizing verified WCAG Contrast levels.

Constraints & System Protocols

- Do not utilize heavy system libraries that cannot compile in lightweight Node environment.
- The manifest.json file must explicitly set the orientation as standalone with start_url targeted to index.html.
- Any external library addition must first undergo a dependency scan for version compatibility.

কৌশলগত নিরাপত্তা এবং রিসোর্স ম্যানেজমেন্ট

অ্যান্ড্রয়েড ফ্যান্টম প্রসেস কিলার (PPK) নিষ্ক্রিয়করণ

অ্যান্ড্রয়েড ১২ বা তার পরবর্তী সংস্করণগুলোতে যুক্ত হওয়া পিপিকে (PPK) মেকানিজম ব্যাকগ্রাউন্ডে থাকা অতিরিক্ত রানটাইম প্রসেসগুলোকে জোরপূর্বক বন্ধ করে দেয়। টারমাক্সে জাভা বা গ্রেডল রান করার সময় যেন হঠাৎ বিল্ড ক্র্যাশ না করে, তা এড়াতে অ্যান্ড্রয়েড ওয়ারলেস ডিবাগিং সেশন চালু রেখে নিচের এডিবি (ADB) কমান্ডগুলো ব্যবহার করা জরুরি:

```
adb shell "/system/bin/device_config set_sync_disabled_for_tests persistent"
adb shell "/system/bin/device_config put activity_manager max_phantom_processes 2147483647"
adb shell settings put global settings_enable_monitor_phantom_procs false
```

টয়বক্স বনাম জিএনইউ (GNU) বাইনারি বৈষম্য মোকাবেলা

টারমাক্সের ভেতরের শিয়ুকু রানটাইম মূলত অ্যান্ড্রয়েডের নিজস্ব সংকুচিত 'টয়বক্স (Toybox)' শেল ব্যবহার করে থাকে। টয়বক্স শেলে জিএনইউ-এর মতো উন্নত ডিরেক্টরি অনুসন্ধান বা ফাইল ফিল্টারিং ক্ল্যাগগুলো (xargs -O বা find -print0) অনুপস্থিত থাকে। এ কারণে যদি কোনো জটিল ডেটা রিডিংয়ের প্রয়োজন হয়, তবে এআই স্ক্রিপ্টকে প্রথমে ফাইলগুলো সাধারণ উপায়ে শেয়ার্ড স্টোরেজে (/sdcard/Download) স্থানান্তর করতে হবে এবং পরবর্তীতে টারমাক্সের নিজস্ব পরিবেশের শক্তিশালী জিএনইউ বাইনারি ব্যবহার করে ফাইল প্রক্রিয়াকরণ সম্পন্ন করতে হবে।

থার্ড-পার্টি এমসিপি বা এআই স্কিলস ব্যবহারে নিরাপত্তা ঝুঁকি

কিছু ওপেন সোর্স এবং আনঅফিসিয়াল জুলস স্কিলস (যেমন: বিভিন্ন অননুমোদিত jules-skills বা গিটহাব ইস্যু ট্রায়াজিং টুলস) ব্যবহার করার ক্ষেত্রে নিরাপত্তার তীব্র ঝুঁকি লক্ষ্য করা গেছে।

- ঝুঁকির ধরন: থার্ড-পার্টি অননুমোদিত রানটাইম স্ক্রিপ্টগুলো গোপনে সিস্টেম কমান্ড রান করতে পারে বা গিটহাব অ্যাকশনের ক্রেডেনশিয়াল ডেটা চুরির চেষ্টা চালাতে পারে।
- সুরক্ষা নীতি: শুধুমাত্র গুগলের অফিসিয়াল এবং সিকিউর এমসিপি ক্লায়েন্ট সংযোগগুলো ব্যবহার করা উচিত। বাহ্যিক কোনো এপিআই কী বা গুরুত্বপূর্ণ সিক্রেট জুলসের ওয়ার্কস্পেসে সরাসরি টেক্সট আকারে না লিখে গিটহাবের সুরক্ষিত ডেটা স্টোরেজ তথা গিটহাব সিক্রেটসের মধ্যে সংরক্ষণ করা সর্বোত্তম সমাধান।

জুলস কোটা এবং রিসোর্স ম্যানেজমেন্ট

জুলস প্ল্যাটফর্মে আপনি দিনে সর্বোচ্চ কয়টি কাজ পরিচালনা করতে পারবেন তা আপনার সাবস্ক্রিপশন প্ল্যানের ওপর নির্ভর করে।

- বিনামূল্যে ব্যবহার (Free Tier): প্রতিদিন সর্বোচ্চ ১৫টি কাজ এবং ৩টি সমসাময়িক কাজ পরিচালনা করা যায়।
- গুগল এআই প্রো (Google AI Pro - \$19.99/mo): প্রতিদিন প্রায় ৭৫টি কাজ এবং ১৫টি সমসাময়িক কাজ পরিচালনা করতে পারে, যা প্রকৌশলী নির্ধারণের ব্যক্তিগত প্রোজেক্টের নিবিড় উন্নয়নের জন্য অত্যন্ত উপযোগী ও পর্যাপ্ত।
- গুগল এআই আল্ট্রা (Google AI Ultra - \$124.99/mo): প্রতিদিন প্রায় ৩০০টি কাজের উচ্চ সুবিধা প্রদান করে।

কৌশলগত সিদ্ধান্ত ও ভবিষ্যৎ নির্দেশনা

মোবাইল ডিভাইস থেকে সম্পূর্ণ কম্পিউটার-বিহীন সফটওয়্যার তৈরির এই স্বয়ংক্রিয় পদ্ধতিটি মূলত একটি বাস্তবমুখী এবং কার্যকর ক্লিপিং। এই সিস্টেমে পোকো এক্স৩ ডিভাইসটির ওপর অতিরিক্ত কম্পাইলেশন লোড না চাপিয়ে সম্পূর্ণ

ক্লাউড আর্কিটেকচার ব্যবহার করা হয়েছে, যা ফোনের কার্যক্ষমতা এবং বিল্ডের গুণমান নিশ্চিত করে। জুলস এবং এমসিপি (Stitch ও Neon) এর গভীর ইন্টিগ্রেশন প্রথাগত এআই কোড জেনারেটরের চেয়ে বহুগুণ শক্তিশালী। এটি কোড জেনারেট করার পর সরাসরি সার্ভারলেস ডেটাবেস টেবিল এবং ইউআই ডিজাইনের মাঝে একটি রিয়েল-টাইম কার্যকরী ব্যাকএন্ড সম্পর্ক স্বয়ংক্রিয়ভাবে নিখুঁতভাবে গড়ে তুলতে পারে। গিটহাব অ্যাকশন রানারে বাবলর্যাপের স্বয়ংক্রিয় ব্যবহার জটিল অ্যান্ড্রয়েড স্টুডিও কোডিংয়ের প্রয়োজনীয়তা দূর করে এবং খুব সহজেই একটি নিখুঁত অ্যান্ড্রয়েড সাইন্ড এপিকে ব্যবহারকারীর ফোনে সরাসরি ডেলিভারি নিশ্চিত করে। সম্পূর্ণ নিরাপত্তা প্রোটোকল মেনে চললে এই সিস্টেমটি প্রকৌশলীর জন্য একটি সম্পূর্ণ স্বাধীন ও মোবাইল-কেন্দ্রিক এআই সফটওয়্যার ডেভেলপমেন্ট ইকোসিস্টেম হিসেবে কাজ করবে।

যেসব কাজের উদ্ধৃতি যোগ করা হয়েছে

1. Jules: Google's Async Coding Agent Is Changing How We Think About AI and Software Development - DEV Community, <https://dev.to/jubinsoni/jules-googles-async-coding-agent-is-changing-how-we-think-about-ai-and-software-development-3j0d> 2. Building Android applications in Termux using Gradle - Grokikipedia, https://grokikipedia.com/page/Building_Android_applications_in_Termux_using_Gradle 3. Changelog | Jules, <https://jules.google/docs/changelog/> 4. Connect MCP clients to Neon - Neon Docs, <https://neon.com/docs/ai/connect-mcp-clients-to-neon> 5. GitHub - google-labs-code/stitch-skills: A library of Agent Skills designed to work with the Stitch MCP server. Each skill follows the Agent Skills open standard, for compatibility with coding agents such as Antigravity, Gemini CLI, Claude Code, Cursor., <https://github.com/google-labs-code/stitch-skills> 6. Design-to-Code with Antigravity and Stitch MCP - Google Codelabs, <https://codelabs.developers.google.com/design-to-code-with-antigravity-stitch> 7. MCP support comes to Jules, <https://jules.google/docs/changelog/2026-02-02/> 8. Complete Guide: Setting Up CI/CD for Android Apps with GitHub Actions, Firebase & Play Store | by Satyam Jha | Medium, <https://medium.com/@satyamkrjha85/complete-guide-setting-up-ci-cd-for-android-apps-with-github-actions-firebase-play-store-5b942208473a> 9. A quick guide to GitHub Actions CI/CD for Android — Firebase Distribution & Play Store : r/androiddev - Reddit, https://www.reddit.com/r/androiddev/comments/1r7l109/a_quick_guide_to_github_actions_cicd_for_android/ 10. Getting started | Jules, <https://jules.google/docs/> 11. Omarbadran37/jules-mcp-server - GitHub, <https://github.com/Omarbadran37/jules-mcp-server> 12. API Reference - Jules, <https://jules.google/docs/api/reference/overview/> 13. Firebase App Distribution Action - GitHub Marketplace, <https://github.com/marketplace/actions/firebase-app-distribution-action> 14. illegalvoid/termux-build-apps: build APKs with Termux, without needing Proot - GitHub, <https://github.com/illegalvoid/termux-build-apps> 15. Firebase App Distribution · Actions · GitHub Marketplace, <https://github.com/marketplace/actions/firebase-app-distribution> 16. jules-skills/automate-github-issues by google-labs-code - High Risk | Security Scan, <https://mondoo.com/ai-agent-security/skills/github/google-labs-code/jules-skills/automate-github-issues/7a76d3bfcf42>